

NBCA-605P: Machine Learning Lab

List of Practicals

Unit-I: Introduction & Statistical Learning Foundations

Practical 1: Train-Test Split and Loss Evaluation

Objective:

To understand the importance of separating data into training and testing sets and to evaluate model performance using Mean Squared Error (MSE). The experiment demonstrates the concepts of training loss, testing loss, and overfitting.

Libraries to be used:

- NumPy
- Pandas
- Scikit-Learn (`train_test_split`, `LinearRegression`, `mean_squared_error`)

Input:

A synthetic or real-world dataset containing independent variables (features) and a continuous target variable.

Expected Output:

- Training dataset and testing dataset sizes.
 - Training Mean Squared Error (MSE).
 - Testing Mean Squared Error (MSE).
 - Interpretation of whether the model is underfitting, well-fitted, or overfitting.
-

Practical 2: Missing Data Imputation

Objective:

To identify missing values in a dataset and apply suitable imputation techniques to prepare data for machine learning algorithms.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (`SimpleImputer`)

Input:

A CSV dataset containing numerical and categorical features with missing values (NaN).

Expected Output:

- Number of missing values in each column.
 - Dataset before imputation.
 - Dataset after imputation.
 - Verification that all missing values have been replaced.
-

Unit-II: Supervised Learning

Practical 3: k-Nearest Neighbours (k-NN) Classification

Objective:

To classify data samples based on the labels of their nearest neighboring data points using a distance-based learning approach.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (`KNeighborsClassifier`)

Input:

The Iris dataset containing flower measurements and corresponding species labels.

Expected Output:

- Predicted class labels for test samples.
 - Classification accuracy.
 - Confusion matrix showing classification performance.
-

Practical 4: Decision Tree Classification

Objective:

To construct a tree-based classification model that makes decisions using hierarchical if-else rules.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (`DecisionTreeClassifier`)
- Matplotlib

Input:

A classification dataset such as Iris, Wine, or a customer classification dataset.

Expected Output:

- Trained decision tree model.
 - Classification accuracy.
 - Visualization of the decision tree structure.
-

Practical 5: Naïve Bayes Classification

Objective:

To implement a probabilistic classification model based on Bayes' theorem and conditional independence assumptions.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (`GaussianNB`)

Input:

A dataset containing multiple features and corresponding class labels.

Expected Output:

- Predicted class labels.
 - Classification accuracy.
 - Probability estimates for each class.
-

Practical 6: Simple Linear Regression

Objective:

To model the linear relationship between an independent variable and a continuous dependent variable.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (`LinearRegression`)
- Matplotlib

Input:

A dataset containing one predictor variable and one target variable (e.g., Hours Studied vs Exam Score).

Expected Output:

- Regression coefficient (slope).

- Intercept.
 - Predicted values.
 - Scatter plot with regression line.
-

Practical 7: Logistic Regression for Binary Classification

Objective:

To build a classification model that predicts binary outcomes using the logistic sigmoid function.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (`LogisticRegression`)

Input:

A binary classification dataset (e.g., Pass/Fail, Disease/No Disease).

Expected Output:

- Predicted class labels.
 - Predicted probabilities.
 - Classification accuracy.
 - Confusion matrix.
-

Unit-III: Ensemble Learning and Support Vector Machines

Practical 8: Voting Classifier Ensemble

Objective:

To combine multiple machine learning classifiers into a single ensemble model and improve classification performance through voting.

Libraries to be used:

- Scikit-Learn (`VotingClassifier`)
- Logistic Regression
- Decision Tree
- Gaussian Naïve Bayes

Input:

A tabular classification dataset.

Expected Output:

- Accuracy of individual classifiers.
 - Accuracy of the voting classifier.
 - Comparison table of results.
-

Practical 9: Random Forest Classification

Objective:

To build an ensemble of decision trees and improve classification performance using bootstrap aggregation.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (`RandomForestClassifier`)
- Matplotlib

Input:

A classification dataset with multiple features.

Expected Output:

- Classification accuracy.
 - Feature importance ranking.
 - Feature importance visualization.
-

Practical 10: Linear SVM Classification

Objective:

To classify linearly separable data using a maximum-margin hyperplane.

Libraries to be used:

- NumPy
- Matplotlib
- Scikit-Learn (`SVC` with linear kernel)

Input:

A binary classification dataset.

Expected Output:

- Classification accuracy.
 - Decision boundary plot.
 - Visualization of support vectors.
-

Practical 11: Nonlinear SVM Classification using RBF Kernel

Objective:

To classify non-linearly separable data using the kernel trick and Radial Basis Function (RBF) kernel.

Libraries to be used:

- NumPy
- Matplotlib
- Scikit-Learn (svc with RBF kernel)

Input:

A non-linear dataset such as the Moons or Circles dataset.

Expected Output:

- Classification accuracy.
 - Nonlinear decision boundary visualization.
 - Kernel-based separation of classes.
-

Unit-IV: Unsupervised Learning and Dimensionality Reduction

Practical 12: K-Means Clustering

Objective:

To group unlabeled data into clusters based on similarity and centroid distance.

Libraries to be used:

- NumPy
- Pandas
- Scikit-Learn (KMeans)
- Matplotlib

Input:

An unlabeled dataset containing multiple numerical features.

Expected Output:

- Cluster assignments.
- Cluster centroids.
- Scatter plot showing clusters and centroids.

Practical 13: DBSCAN Density-Based Clustering

Objective:

To discover clusters of arbitrary shapes and identify outliers using density-based clustering.

Libraries to be used:

- NumPy
- Scikit-Learn (DBSCAN)
- Matplotlib

Input:

A dataset containing irregular cluster structures and noise.

Expected Output:

- Cluster labels.
- Identification of noise points.
- Scatter plot showing clusters and outliers.

Practical 14: Principal Component Analysis (PCA)

Objective:

To reduce the dimensionality of high-dimensional datasets while preserving maximum variance.

Libraries to be used:

- Pandas
- NumPy
- Scikit-Learn (PCA)
- Matplotlib

Input:

A dataset containing multiple correlated features.

Expected Output:

- Principal components.
 - Explained variance ratio.
 - Two-dimensional visualization of transformed data.
-

Unit-V: Neural Networks and Deep Learning

Practical 15: Multilayer Perceptron (MLP) Classification using Keras

Objective:

To design, train, and evaluate a feed-forward Artificial Neural Network for classification tasks using Keras and TensorFlow.

Libraries to be used:

- TensorFlow
- Keras (`Sequential`, `Dense`)
- NumPy
- Pandas

Input:

A binary or multi-class classification dataset.

Expected Output:

- Neural network architecture summary.
- Training loss and accuracy for each epoch.
- Final test accuracy.
- Predicted class labels for unseen samples.

Practical 16: Multilayer Perceptron (MLP) with Backpropagation

Objective

To implement a Multi-Layer Perceptron (MLP) neural network and understand how the Backpropagation algorithm updates network weights by propagating prediction errors from the output layer back through hidden layers during training.

Libraries to be used

- TensorFlow
- Keras (`Sequential`, `Dense`)
- NumPy
- Pandas
- Matplotlib (optional for loss visualization)

Input

A supervised classification dataset consisting of:

- Feature matrix $\mathbf{X}$

- Target class labels $\mathbf{y}$

Examples:

- Iris Dataset (3-class classification)
- MNIST Digit Dataset
- Customer Classification Dataset

Expected Output

- Display of the neural network architecture.
- Training logs showing:
 - Epoch number
 - Training Loss
 - Training Accuracy
- Visualization of loss reduction across epochs.
- Final classification accuracy on test data.
- Predicted class labels for unseen samples.
- Demonstration that loss decreases as weights are updated through Backpropagation.